

Iterators em .Js

#programming

#javascript

Em JavaScript, **iterators** são objetos responsáveis por **percorrer uma sequência de valores passo a passo**. Eles controlam o estado da iteração e decidem qual é o próximo valor.

Video sobre:

[Iterators in JavaScript using Quokka.js](#)

1. Ideia central

Um iterator é um objeto com um método:

```
next()
```

Cada chamada avança a sequência e retorna:

```
{ value: ..., done: true/false }
```

- `value` → valor atual
 - `done` → indica se terminou
-

2. Exemplo básico

```
const arr = [1, 2, 3];  
const it = arr[Symbol.iterator]();  
it.next(); // { value: 1, done: false }  
it.next(); // { value: 2, done: false }  
it.next(); // { value: 3, done: false }  
it.next(); // { value: undefined, done: true }
```

3. Como funciona

O iterator mantém internamente:

- a posição atual
- a lógica de avanço

Cada `next()`:

1. retorna o valor atual
2. atualiza o estado interno
3. indica se ainda há elementos

4. Relação com iterables

Um iterator normalmente vem de um **iterable**:

```
arr[Symbol.iterator]()
```

- `iterable` → cria o iterator
- `iterator` → percorre os valores

5. Uso automático

```
for (const x of arr) { console.log(x);}
```

Esse loop usa o iterator internamente, chamando `next()` até `done === true`.